

스스로 판단하고 행동하는
지능형 비서, Agent

스스로 판단하고 행동하는 지능형 비서, Agent

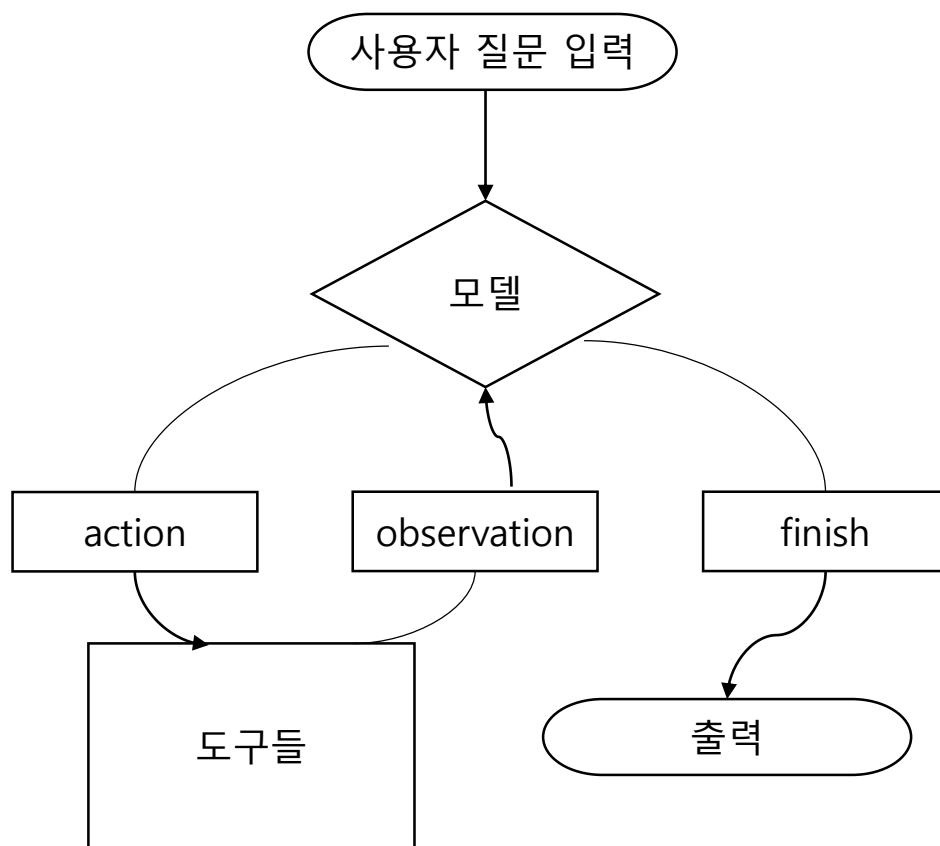
1. 도구 이해하기
2. 에이전트 첫걸음
3. ReAct 패턴 이해하기
4. 외부 도구 사용하기

1. 도구 이해하기

- “만약 LLM이 스스로 생각하고, 상황에 따라 필요한 도구를 찾아 사용하며, 복잡한 요청을 처리할 수 있다면 어떨까?”
- 이번 장에서 다룰 주요 내용이 바로, 이 질문에 대한 답이 될 에이전트(agent)
- agent는 LLM을 두뇌로 사용하여, 우리에게 주어진 도구(tool) 목록 중에서 가장 적절한 것을 선택하고 실행하는 지능형 비서
- agent의 가장 기본적인 구성 요소인 도구를 만들고, 이 도구를 agent에 연결하는 방법을 알아봄
- AI 모델이 사용할 도구란?
 - 우리의 목표는 agent에게 도구들을 제공했을 때, 적재적소에 도구를 사용해 문제를 해결하도록 하는 것
 - 우리가 LMM에게 질문을 던졌을 때, 항상 만족스러운 답변을 받을 수 있는 것은 아님
 - 가끔은 사용자 질문을 이해하지 못하거나 혹은 잘못된 정보를 사실인 것처럼 내놓을 때도 있음
 - 이런 상황이 우리 서비스의 메인 기능에서도 동일하게 발생한다면 매우 곤란
 - 이런 상황을 대비하려면 agent가 적절한 기능을 수행할 수 있도록 도구를 만들어서 제공해야 함
 - 즉, 특정한 상황이 발생한다면, agent가 적절한 도구를 호출해 정해진 답변을 반환하도록 만드는 것
 - 입력이 주어질 때, 정해진 역할을 수행한 다음, 완성된 출력을 반환하는 것을 의미 - 즉, AI의 도구란 함수의 개념과 동일

1. 도구 이해하기

- 그럼 agent는 이 함수들을 어떻게 구분하여 사용할 수 있을까?
- 어떻게 스스로 판단하고 적절한 함수를 호출하고 이를 통한 답변을 활용하는 것일까?



- 에이전트의 도구 사용 과정

1. 도구 이해하기

- 에이전트의 도구 사용 과정

1. 사용자 질문 입력 -> 모델

- 사용자의 입력이 모델에게 전달

2. 모델 -> 도구들(action)

- 사용자의 질문을 확인한 모델이 '지금은 도구를 써야겠다'고 판단
- 이때, 어떤 도구를 어떤 인자로 호출할지 action을 생성. 여기서 action은 실제 도구 호출 과정
- 주어진 도구들이 각자 어떤 역할을 수행하는지 확인하고, 현재 질문 내용을 토대로 도구를 사용할 것인지, 아니면 직접 답변을 생성할 것인지 결정

3. 도구들 -> 모델(observation)

- 도구가 실행된 뒤, 그 결과값이 반환. 이 결과값을 랭체인에서는 observation이라고 정의함
- 도구가 모델을 '관측'하는 것이 아니라, '모델이 도구의 실행 결과를 관측한다'고 이해하면 됨. 그림에서는 이 '도구 결과 -> 모델로 돌아가는 데이터 흐름'을 observation이라고 표시해 둠

4. 모델 -> 출력(finish)

- 모델은 '입력 + 하나 혹은 여러 번의 observation'을 토대로 이제 도구를 더 쓰지 않겠다고 결정하면 최종 답변을 생성함

1. 도구 이해하기

- 가장 단순한 도구 만들기

- 앞선 과정에서 확인할 수 있듯이 도구를 만든다는 것은 결국, 파이썬 함수를 하나 정의하는 것
- 이렇게 정의된 함수를 랭체인에서 제공하는 데코레이터를 사용하여 도구라고 선언하는 것으로 완성됨
- 가장 기초적인 도구를 하나 만들어 보면서 이해도를 높여 봄
- 패키지 설치 및 확인

```
(.venv)  
$ pip install langchain langchain-core langchain-openai langchain-community  
$ pip show langchain
```

1. 도구 이해하기

● 가장 단순한 도구 만들기

• 도구 정의

- 패키지 설치를 모두 마쳤다면, `langchain_core.tools`의 `tool` 데코레이터를 불러옴
- 그리고 아주 단순한 형태의 함수를 하나 정의하고, 그 위에 데코레이터를 부착
- 단, 주의할 점은 도구로 선정할 함수들은 반드시 `docstring`을 추가해야 함
- 함수의 역할과 기능을 명확히 설명할 수 있는 내용을 작성해 주어야 에이전트가 도구를 호출할 것인지 결정할 수 있음

```
from langchain_core.tools import tool

@tool
def greet(name):
    """사용자의 이름을 받아 인사합니다."""
    return f"안녕 {name}!"

print(greet.invoke({"name": "철수"}))
```

- `greet` 함수를 정의하고 `tool` 데코레이터를 이용해 도구로 선언
- 이렇게 도구가 된 함수는 `Runnable` 객체가 되어 `invoke` 메서드를 통해 호출할 수 있게 됨
- 우리가 직접적으로 이 도구를 호출하는 경우는 거의 없지만 이해를 위하여 `invoke` 메서드를 호출한 결과를 확인해 봄
- `tool` 데코레이터는 함수를 일반적인 반환값이 아닌 랭체인을 위한 올바른 흐름을 위한 데이터를 반환할 수 있도록 꾸며 줌

1. 도구 이해하기

● 도구 메타데이터 다듬기

- 이제 tool 데코레이터를 잘 활용할 수 있는 방법에 대해 알아봄
- tool 데코레이터는 단순히 함수를 도구로 정의하는 용도를 넘어서서, 더 명확히 어떠한 도구인지를 정의하는 데 도움을 줌
- 정의한 함수의 이름, 설명, 반환하여야 할 데이터의 형태 등을 정의할 수 있음
- 도구에 상세한 이름 부여하기
 - 기본적으로 에이전트는 함수를 선언할 때 작성한 이름을 도구 목록에 담아서 사용함
 - tool 데코레이터의 첫 번째 인자로 함수의 이름을 명시적으로 표기하여 도구에 대한 상세한 이름 부여 가능

```
@tool("math_calculator")
def calculator(expression: str) -> str:
    """수학 계산이 필요할 때만 사용하세요."""
    return str(eval(expression))

print(calculator.name) # math_calculator
print(calculator.description) # 수학 계산이 필요할 때만 사용하세요.
print(calculator.invoke({"expression": "2 + 3 * 4"})) # 14
```


1. 도구 이해하기

● 도구 메타데이터 다듬기

• 도구에 설명 추가하기

- docstring로 함수를 설명하고, 이를 에이전트에게 전달하여 무엇을 위한 도구인지를 알려줄 수 있음
- 다만, 이는 본래 docstring의 용도와는 다름
- 일반적으로 함수 내부에 작성하는 docstring은 보통 개발자용 설명(내부 구현, 예외, 반환 타입등)임
- 에이전트에게 전달해야 하는 내용(이 도구는 언제 써야 하는지, 입력으로 무엇을 받는지)과는 다소 차이가 있음
- 따라서, 에이전트만을 위한 설명을 tool 데코레이터의 description 인자를 통해 전달할 수 있음

```
@tool("math_calculator", description="표현식(문자열)을 계산하여 반환")
def calculator(expression):
    """
    간단한 수학 표현식을 계산합니다.
    Args:
        expression (str): 계산할 수학 표현식(예: "2 + 3 * 4")
    Returns:
        str: 계산 결과
    """
    return str(eval(expression))

print(calculator.description) # 표현식(문자열)을 계산하여 반환.
print(calculator.invoke({"expression": "2 + 3 * 4"})) # 14
```

1. 도구 이해하기

● 도구 메타데이터 다듬기

- Pydantic 스키마로 구조화된 입력 받기

- 도구에 입력할 데이터의 형태를 제어하고자할 때 활용할 수 있는 것이 바로 Pydantic 라이브러리

1) Pydantic 라이브러리

- Pydantic 라이브러리의 핵심은 데이터 구조의 유효성 검사와 설정 관리
- 라이브러리에서 제공하는 BaseModel을 상속한 클래스로 데이터의 구조(schema)를 정의할 수 있음
- 데이터 구조를 정의할 때는 각 필드(field)에 타입 힌트를 부여함으로써 유효성 검사와 JSON 구조를 생성할 수 있게 됨

```
from pydantic import BaseModel

class SearchInput(BaseModel):
    query: str
    top_k: int = 5
```

- SearchInput 클래스를 예시 코드와 같이 정의하면 {"query": 123}와 같은 정수 데이터가 들어와도 문자열로 변환하거나, 변환조차 불가능하면 에러를 반환함

1. 도구 이해하기

● 도구 메타데이터 다듬기

2) 도구에서 구조화된 데이터 입력받기

- 도구에서 입력받을 데이터를 Pydantic으로 구조화시켜 받을 수 있도록 하겠음
- 먼저 도시와 단위 정보를 반환하는 도구 정의. 대신, 실제 날씨와 단위 정보를 위해서는 날씨 관련 API가 필요하므로, 코드 구성에만 집중

```
from pydantic import BaseModel, Field

class WeatherArgs(BaseModel):
    city: str = Field(description="도시 이름")
    units: str = Field(description="섭씨 또는 화씨", default="섭씨")

@tool(args_schema=WeatherArgs)
def get_weather_units(city, units="섭씨"):
    """해당 도시에서 주로 사용하는 온도 단위를 반환합니다."""
    return f"{city} (은/는) 주로 {units} 단위를 사용합니다."

print(get_weather_units.invoke({"city": "서울"})) # 서울 (은/는) 주로 섭씨 단위를 사용합니다.
```

- BaseModel 클래스를 상속받은 WeatherArgs 클래스를 정의
- 도시와 단위 정보 모두 문자열을 필요로 하므로 타입 힌트는 str로 작성.
- 각 필드는 Field 클래스를 사용하여 무엇을 위한 필드인지 설명(description)하거나 기본값(default)을 정의할 수 있음
- 함수를 정의하고 데코레이터를 사용해 도구로 등록 - args_schema 인자에 WeatherArgs 클래스를 전달함으로써 입력받을 데이터의 구조를 설정할 수 있음
- 함수 매개변수를 정의하는데, 이 매개변수명은 WeatherArgs 클래스에 정의한 필드명과 일치하여야 함

2. 에이전트 첫걸음

- 단일 에이전트 생성하기

- 에이전트의 가장 중요한 역할

- 사용자의 질문을 통해, 어떤 도구가 필요한지 판단하고, 도구 실행 결과를 확인하고, 완료할 것인지 도구의 사용을 반복할 것인지 스스로 결정한다는 것
 - 우선 도구 없이 에이전트만 생성하고 활용해 보도록 함

- 오픈AI API key 등록

```
from dotenv import load_dotenv  
  
load_dotenv()
```

- 에이전트를 만들기 위해서는 두뇌 역할을 할 모델 선정
 - 이를 위해 오픈AI API key를 환경 변수 읽어 옴

2. 에이전트 첫걸음

- 단일 에이전트 생성하기
 - 모델 지정과 에이전트 생성

```
from langchain.agents import create_agent
from langchain_openai import ChatOpenAI

# LLM 준비
model = ChatOpenAI(model="gpt-5-nano")

# Agent 생성
agent = create_agent(
    model=model,
    system_prompt="당신은 도움이 되는 AI 어시스턴트입니다. 사용자의 질문에 정확하게 답하세요."
)
```

- model 변수에 'gpt-5-nano' 모델을 불러와 할당
- langchain.agents 모듈에서 create_agent 함수를 불러와 에이전트 정의
- gpt 모델과 시스템 프롬프트를 작성하여 system_prompt 인자로 전달
- 랭체인에서 제공하는 에이전트 생성 완료

2. 에이전트 첫걸음

- 단일 에이전트 생성하기

- 에이전트 실행하기

```
result = agent.invoke({
    "messages": [{"role": "user", "content": "5 × 12는?"}]
})

print(result["messages"][-1].content) # 60입니다. 5 × 12 = 60.
```

- 에이전트에게 사용자의 질문을 전달하는 방법은 LCEL 방식의 체인을 호출하는 것과 동일하게 invoke 메서드
 - 체인의 경우 프롬프트에 작성된 변수를 키로, 사용자의 질문을 값으로 전달했다면, 에이전트의 경우 정확히 누가(role), 어떤 내용(content)을 어떻게(message) 전달하는지 명확히 해주어야 함
 - 반환값 역시 복잡한 구조를 가지고 있어 이번에는 모델의 답변만 추출하여 출력해 보겠음

2. 에이전트 첫걸음

- 도구를 활용하는 에이전트 생성하기

- 도구를 포함한 에이전트 생성하기

- 앞선 실습의 calculator 함수와 get_weather_units 함수를 정의하고, 도구로 등록한 것을 에이전트에 포함 시키기
 - 방법은 create_agent 함수를 호출할 때 tools 인자를 추가해 주기만 하면 됨
 - 각 함수들을 리스트에 담아 인자로 전달
 - 현재 생성한 도구들이 매우 단순한 형태인 만큼 시스템 프롬프트에 도구 사용을 강제하는 문구를 추가하겠음

```
agent = create_agent(  
    model=model,  
    tools=[calculator, get_weather_units],  
    system_prompt="""  
        당신은 도움이 되는 AI 어시스턴트입니다. 사용자의 질문에 정확하게 답하세요.  
        상황에 맞는 도구를 반드시 사용하여 답변합니다.  
    """)
```

2. 에이전트 첫걸음

- 도구를 활용하는 에이전트 생성하기

- 에이전트 실행하기

- 우리가 만든 도구들을 제대로 사용하는지 확인하기 위해 총 3번 invoke 메서드 실행

- 1) 수학 계산 질문

```
result = agent.invoke({  
    "messages": [{ "role": "user", "content": "5 × 12는?" }]  
})  
  
print(result["messages"][-1].content) # 5 × 12의 값은 60입니다.
```

- 실행 결과는 “5 × 12의 값은 60입니다.”
 - 도구를 사용한 것인지 명확하게 느껴지지 않는

2. 에이전트 첫걸음

- 도구를 활용하는 에이전트 생성하기

- 에이전트 실행하기

- 우리가 만든 도구들을 제대로 사용하는지 확인하기 위해 총 3번 invoke 메서드 실행

- 2) 미국의 온도 단위를 묻는 질문

```
result = agent.invoke({
    "messages": [{
        "role": "user",
        "content": "미국은 어떤 온도 단위를 주로 사용하나요?"
    }]
})

print(result["messages"][-1].content)
```

미국은 주로 화씨(°F) 단위를 사용합니다. 다만 과학 연구나 국제 상황에서는 섭씨(°C)도 가끔 사용됩니다.

- 앞서 정의한 도구 get_weather_units의 반환값과는 완전히 다른 형태의 답변이 생성됨

2. 에이전트 첫걸음

- 도구를 활용하는 에이전트 생성하기

- 에이전트 실행하기

- 우리가 만든 도구들을 제대로 사용하는지 확인하기 위해 총 3번 invoke 메서드 실행

- 3) 랭체인이란 무엇인가를 묻는 질문

```
result = agent.invoke({
    "messages": [{
        "role": "user",
        "content": "랭체인이란 무엇인가요?"
    }]
})

print(result["messages"][-1].content)
```

랭체인(LangChain)은 대형 언어 모델(LLM)을 이용한 애플리케이션을 쉽게 만들 수 있도록 도와주는 오픈 소스 프레임워크입니다. LLM만으로는 부족한 부분들을 체인처럼 엮어주고, 외부 데이터/도구와의 연동을 간편하게 처리할 수 있게 해줍니다.

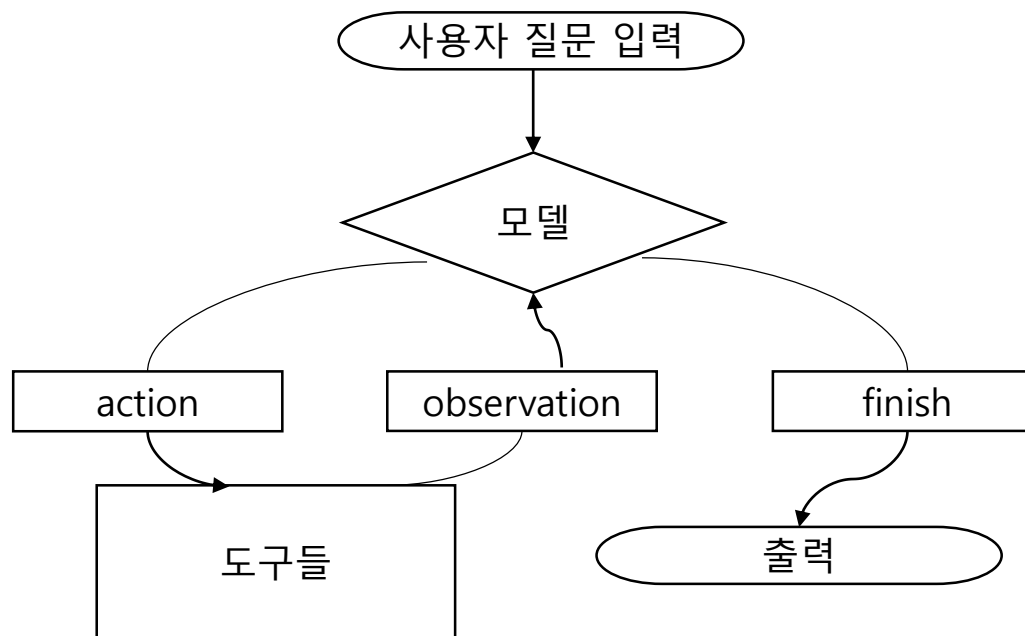
... (생략) ...

- 두뇌 역할인 모델 'gpt-5-nano'가 랭체인에 대한 설명을 잘 해주고 있음
 - 랭체인과 관련된 도구를 전달한 적은 없으니, 이번 답변을 순수하게 모델에 의해 생성된 것임

3. ReAct 패턴 이해하기

- ReAct란?

- ReAct(추론(reasoning)+행동(acting))는 프린스턴 대학의 야오(Yao)등이 2022년 발표한 프롬프트 엔지니어링 패러다임
- 랭체인(Agent)의 에이전트는 이 패러다임을 실제 코드로 구현한 것
- 핵심은 LLM이 단순히 도구를 호출하는 것이 아니라, 왜 그 도구를 선택했는지 추론 과정을 명시적으로 표현하고, 도구의 결과를 해석해 최종 답변을 재구성한다는 점
- 에이전트는 이 과정을 반복함



- 에이전트의 도구 사용 과정

3. ReAct 패턴 이해하기

- ReAct란?

- 에이전트의 도구 사용 과정을 앞선 예시로 더 상세하게 추론 과정을 살펴보자면

- 사용자: “미국은 어떤 온도 단위를 주로 사용하나요?”



- [Reasoning] LLM: “get_weather_units 도구를 사용해야겠다.”



- [Acting] 도구 실행: get_weather_units(city=“미국”, units=“화씨”)



- [Observation] 결과: “미국 (은/는) 주로 화씨 단위를 사용합니다.”



- [Reasoning] LLM: “미국은 주로 화씨(Fahrenheit, °F) 단위를 사용합니다. 참고로 ...”



- [finish] LLM: “이제 답변 할 수 있다.”



- 최종 답변: “미국은 주로 화씨 (Fahrenheit, °F) 단위를 사용합니다. 참고로 ...”

3. ReAct 패턴 이해하기

● ReAct란?

- 에이전트는 도구를 통해 얻어낸 답변을 그대로 사용자에게 반환하지 않음
- 한 번 더 추론 과정을 거쳐, 도구의 결과를 정리하고 맥락을 더해 유려한 최종 답변을 제공
- 이렇게 만들어지는 답변의 스타일은 사용한 LLM에 따라 서로 다른 양상을 보일 것
- 그래서 “도구를 활용하는 에이전트 생성하기” 절의 결과가 도구의 사용 없이 ‘gpt-5-nano’ 모델이 생성한 것처럼 느껴진 것

3. ReAct 패턴 이해하기

● ReAct 과정 관찰하기

- 실제로 에이전트가 어떻게 실행되는지 실시간으로 관찰해 보고자 함
- 이를 위해, stream 메서드를 사용함
- 이전에는 단순히 stream 메서드를 호출해 얻은 결과 chunk를 출력만 해보았다면, 이번에는 에이전트가 반환해 주는 다양한 속성을 하나씩 확인해 보며 활용해 봄

• YAPF 패키지

- stream 메서드를 사용하기 앞서 구글 개발자들이 만든 오픈 소스 프로젝트인 YAPF 패키지를 설치
- 에이전트가 반환하는 값의 구조는 매우 많은 정보를 가지고 있는데 반해, 출력으로 제대로 확인하기 어려움
- YAPF가 제공하는 FormatCode를 사용하면, 이러한 불편함을 줄일 수 있음

```
(.venv)  
$ pip install yapf
```

3. ReAct 패턴 이해하기

- ReAct 과정 관찰하기

- 스타일 설정

```
from yapf.yapflib.yapf_api import FormatCode  
  
style = {'COLUMN_LIMIT': 50}
```

- 패키지 설치가 끝난 후, FormatCode 클래스를 불러오고, 사용할 스타일을 지정
 - COLUMN_LIMIT 키 값에 작성할 정수는 실습 환경에 맞춰 조절하면 됨
 - 이렇게 설정한 style은 FormatCode 클래스로 변환할 때 넘겨주어 적용할 수 있음
 - <https://github.com/google/yapf> 참조

3. ReAct 패턴 이해하기

- ReAct 과정 관찰하기

- stream 메서드

```
query = "미국은 어떤 온도 단위를 주로 사용하나요?"
messages = [{
    "role": "user",
    "content": query
}]

for chunk in agent.stream(
    {"messages": messages},
    stream_mode="updates", # 에이전트 진행 상황 단위 스트림
):
    text, flag = FormatCode(str(chunk), style_config=style)
    print(text)
```

- stream 메서드를 사용하는 방법은, invoke 메서드를 사용할 때와 같이 에이전트에게 넘겨줄 질문을 받는 것으로 시작
- 다양한 도구를 활용하는지 확인하고 싶으니, query 변수와 messages 변수를 따로 할당해 두어 수정하기 편하게 관리
- stream_mode 인자를 활용하여 현재 상태를 어떻게 보여주도록 할지도 설정할 수 있는데, 기본값인 “update”를 사용하여 변경 사항에 대해서만 출력하도록 함. 만약, 전체 상태를 포함한 딕셔너리를 확인하고 싶다면, 이 값을 “values”로 변경하면 됨
- chunk 값을 FormatCode로 포맷팅하기 위해 문자열로 변환하여 전달

3. ReAct 패턴 이해하기

● ReAct 과정 관찰하기

- stream 메서드
 - AIMessage

```
AIMessage(  
    content="",  
    ...  
    response_metadata={  
        ...  
        'model_provider': 'openai',  
        'model_name':  
        'gpt-5-nano-2025-08-07',  
        'system_fingerprint': None,  
        'service_tier': 'default',  
        'finish_reason': 'tool_calls',  
        'logprobs': None  
    },  
    tool_calls=[{  
        'name': 'get_weather_units',  
        'args': {  
            'city': '미국'  
        },  
    },  
    ...
```

- 모델의 답변이 항상 content를 가지고 있지는 않음
- 추론 과정에서 최종 결정이 도구를 호출하는 것이라면, content는 비어 있을 수 있음
- 첫 번째 AIMessage 객체는 도구 'get_weather_units'를 호출했기 때문에 별도의 content 값이 없음
- 최종 결정이 어떻게 되었는지는 'finish_reason'를 통해 알 수 있음
- 이번에는 도구를 호출했으므로 'tool_calls' 문자열이 할당되어 있음
- 이때, 어떤 도구를 호출했는지는 tool_calls 리스트에 작성되어 있음
- 그 외에도 모델 정보, 메타데이터 등도 확인할 수 있어 디버깅 시 활용 가능

3. ReAct 패턴 이해하기

- ReAct 과정 관찰하기

- stream 메서드
 - ToolMessage

```
ToolMessage(  
    content='미국 (은/는) 주로 섭씨 단위를 사용합니다.',  
    name='get_weather_units',  
    ...  
)
```

- ToolMessage 역시 content를 가짐
- 도구를 호출하여 반환된 값이 content에 할당됨
- 즉, 서로 다른 객체들(AI, Tool, Human) 모두 유사한 구조를 가지고 있다는 뜻
- 모델, 사용자, 도구의 반환값은 모두 이 content를 통해 전달되므로 구조를 한 번만 확인해 두면 되겠음
- 현재 'get_weather_units' 도구는 항상 고정된 값을 반환함
- 그 결과 “미국 (은/는) 주로 섭씨 단위를 사용합니다”와 같이 화씨가 아닌 섭씨를 반환했음
- units 인자의 기본값이 ‘섭씨’였기 때문

3. ReAct 패턴 이해하기

- ReAct 과정 관찰하기

- stream 메서드
 - 다시 AIMessage

```
AIMessage(  
    content=  
    '미국은 일반적으로 화씨(Fahrenheit, °F) 단위를 주로 사용합니다. ...(생략)...',  
    ...  
    response_metadata={  
        'token_usage': {  
            ...  
            'finish_reason': 'stop',  
            ...  
        },  
        ...  
    },  
    ...  
)
```

- 도구의 반환값을 토대로 모델은 다시 답변을 생성함.
- 잘 다듬은 문장을 content에 할당한 것을 볼 수 있음. 도구가 반환한 값을 그대로 사용하지 않고 적절한 값으로 수정하고 부연 설명을 추가한 것
- 'finish_reason'의 값도 'stop'으로 변경되었음. 더 이상 도구를 호출할 필요없이 답변을 마무리하겠다고 결정

3. ReAct 패턴 이해하기

● ReAct 과정 관찰하기

- stream 메서드 출력 정리
 - 에이전트가 반환하는 값의 구조를 지금까지 chunk의 값을 모두 출력하는 것은 바람직하지 못함
 - stream 과정에서 현재 반환된 객체가 무엇인지(모델, 사용자, 도구)에 따라 조건 분기하고, 각 상황에 따라 말머리와 함께 출력하도록 작성
 - 대신, 이번에는 math_calculator와 get_weather_units 도구를 모두 사용할 수 있도록 질문을 변경하여 에이전트를 호출

```
query = "5+2를 계산하고, 미국은 어떤 온도 단위를 주로 사용는지도 알려주세요."
messages = [{
    "role": "user",
    "content": query
}]

for chunk in agent.stream(
    {"messages": messages},
    stream_mode="values", # 에이전트 진행 상황 단위 스트림
):
    ... (생략) ...

    elif latest_msg.__class__.__name__ == "ToolMessage":
        print(f"[Observation] {latest_msg.content}")
```

3. ReAct 패턴 이해하기

- ReAct 과정 관찰하기

- stream 메서드 출력 정리

```
[Reasoning] 도구 호출: math_calculator
[Acting] math_calculator({'expression': '5+2'})
[Reasoning] 도구 호출: get_weather_units
[Acting] get_weather_units({'city': 'United States', 'units': '화씨'})
[Observation] United States (은/는) 주로 화씨 단위를 사용합니다.
[Finish] - 5 + 2 = 7 입니다.
- 미국은 주로 화씨(Fahrenheit) 단위를 사용합니다. 다만 과학 연구나 특정 분야에서는 섭씨(Celsius)도 사용될 수 있습니다. 예: 0°C는 약 32°F
입니다.
```

- 이번 호출에서는 get_weather_units 도구를 사용하면서 units 인자에 '화씨' 값을 삽입했음
 - 수학 계산을 위한 도구도 잘 호출한 것을 볼 수 있음. 이때 호출한 도구의 이름이 math_calculator
 - 도구로 등록할 때, tool 데코레이터의 첫 번째 인자로 작성했던 도구 이름이 잘 반영된 모습
-
- 추론, 행동, 관찰, 완료를 기준으로 조건 분기하여 출력해 봄

4. 외부 도구 사용하기

- 지금까지 도구를 생성, 등록하고 에이전트에게 전달하여 어떤 답변을 내놓는지 그 추론 과정까지 살펴봄
- 랭체인은 자체적으로 제공하는 기능 외에도 각종 통합 기능을 지원함
- ‘어떤 기능이 이미 만들어져 있는지 파악하고, 필요할 때 적절히 가져다 쓰는 판단력’을 갖추는 것이 중요
- 랭체인 통합 패키지 목록의 공식 문서
 - <https://docs.langchain.com/oss/python/integrations/providers/overview> 참조

4. 외부 도구 사용하기

- 현재 LLM 모델(우리가 사용하고 있는 ‘gpt-5-nano’ 포함)은 일반적인 질문에 대해서는 이미 충분히 훌륭한 답변을 제공하고 있음
- 하지만, 특정 주제와 관련된 영상을 추천해 달라는 질문에는 썩 만족스러운 답변을 내놓지는 못함
- 동일한 질문을 계속해서 반복하면 어쩌다 한번 유튜브 영상 링크를 가져올 때도 있지만, 정확한 정보라고 보기는 어려움
- 이러한 상황이 지속된다면 사용자 경험은 낮아지기만 할 것임
- 이럴 때 우리는 외부 도구의 힘을 빌릴 수 있음.
- 에이전트가 정확한 영상 정보를 가져오도록 도구를 전달해 주는 것
 - YouTubeSearchTool 도구를 사용해 보도록 함

4. 외부 도구 사용하기

- YouTubeSearchTool

- 패키지 설치(업데이트하지 않아도 됨)

```
(.venv)  
$ pip install youtube-search
```

- 도구 준비

- 커뮤니티로 제공되는 도구들은 langchain_community.tools에서 불러올 수 있음
- 이곳에서 YouTubeSearchTool를 불러오고, 객체를 생성하는 것으로 도구 준비는 끝
- 함수를 정의하고, 데코레이터로 도구 등록하는 과정은 패키지에서 이미 모두 처리되어 있으므로, 우리는 에이전트에게 전달하기만 하면 됨

```
from langchain_community.tools import YouTubeSearchTool  
  
youtube_tool = YouTubeSearchTool()
```


4. 외부 도구 사용하기

- YouTubeSearchTool

- 에이전트 준비
 - 에이전트를 위해 어떤 모델을 사용할 것인지, 정확한 역할은 무엇인지를 정해야 함

```
from langchain.agents import create_agent
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5-nano", temperature=0)

media_system_prompt = (
    """
    당신은 한국어 미디어 큐레이터입니다.
    질문을 받으면 youtube_tool로 최신 영상 정보를 찾고,
    주제에 대해서 간략한 의견을 남겨주세요.
    """
)

media_agent = create_agent(
    model=model,
    tools=[youtube_tool],
    system_prompt=media_system_prompt,
)
```

4. 외부 도구 사용하기

● YouTubeSearchTool

- 답변 생성
 - 모든 준비를 마쳤으니 에이전트에게 질문하고, 답변을 받아보겠음
 - stream 메서드를 사용해 스텝별로 내용만 확인하겠음. 이를 위해 chunk 딕셔너리가 가진 키와 벨류를 각각 step_name과 state로 구분하여 출력했음

```
query = "와인 초보자를 위한 가이드 영상 찾아줘"  
messages = [{"role": "user", "content": query}]  
  
for chunk in media_agent.stream(  
    {"messages": messages},  
    stream_mode="updates",  
):  
    for step_name, state in chunk.items():  
        # step_name: "model" / "tools" 같은 노드 이름  
        print("step:", step_name)  
        # 마지막 메시지만 보기  
        last_msg = state["messages"][-1]  
        print("messages:", last_msg.content)
```

4. 외부 도구 사용하기

● YouTubeSearchTool

• 답변 생성

```
step: model
messages:
step: tools
messages: ['https://www.youtube.com/watch?v=SJ86bgH4yfU&pp=ygUa7JmA7J24IOy0i0uzt0yekCDqsIDsnbTrk5w%3D',
'https://www.youtube.com/watch?v=X62EISjNQ5M&pp=ygUa7JmA7J24IOy0i0uzt0yekCDqsIDsnbTrk5w%3D']
step: model
messages: 다음 두 편의 영상이 검색 결과로 나왔습니다. 와인 초보자를 위한 기초 가이드를 다루는 콘텐츠로 보입니다.
```

recommended 영상

- <https://www.youtube.com/watch?v=SJ86bgH4yfU&pp=ygUa7JmA7J24IOy0i0uzt0yekCDqsIDsnbTrk5w%3D>
- <https://www.youtube.com/watch?v=X62EISjNQ5M&pp=ygUa7JmA7J24IOy0i0uzt0yekCDqsIDsnbTrk5w%3D>

간단한 의견

- 두 영상 모두 와인 초보자가 알아야 할 기본 개념(와인 종류의 차이, 테이스팅의 기본, 보관/서빙 팁 등)을 다룰 가능성이 큼니다. 길이가 비교적 짧아 바로 따라 하며 배우기에 좋습니다.
- 한국어 자막이나 한국 채널의 영상일 가능성이 있어 초보자 입장에서 이해하기 쉬울 수 있습니다. 다만 영상 제목과 채널 신뢰도는 YouTube 페이지에서 확인해 보시는 걸 권장합니다.

다음 단계 제안

- 더 구체적으로 원하시면: (1) 한국어 채널 우선, (2) 길이 5-12분 정도의 초보용 영상, (3) 보관/서빙/페어링 같은 특정 주제별 영상 등으로 조건을 주시면 추가로 후보를 더 찾아 큐레이션해 드릴게요.
- 원하시면 이 두 편을 시작으로 초보자용 3편짜리 추천 재생목록으로 정리해 드리겠습니다.

4. 외부 도구 사용하기

● YouTubeSearchTool

- 답변 생성
 - 한 번의 요청으로 한 번의 도구 호출이 있었음. 이 실행 결과는 모두 다를 수 있음
 - 에이전트가 충분히 만족스러운 영상 링크가 모이지 않았다고 판단했다면 여러 영상을 수집하는 모습을 보일 수 있음
 - 도구를 사용하지 않고 수행한 에이전트와 비교하면 차이를 확실히 느낄 수 있음
 - 도구를 사용하지 않았을 때 영상 추천 링크를 받았을 수도 있지만, 그렇게 생성된 링크는 대부분 유튜브에 직접 특정 키워드를 검색한 결과였을 것
 - 반면, YouTubeSearchTool은 명확히 특정 영상의 링크를 추천하고 있음. 일반적인 사용자들이 바라는 추천 서비스에 가깝다고 볼 수 있음